

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

★ OVERVIEW

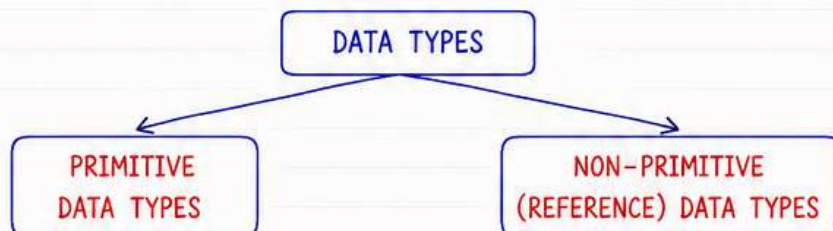
Java is a strongly typed language. It uses data types to define the type of data a variable can hold.

Java data types are divided into two categories.

- Primitive Data Types
- Non-Primitive (Reference) Data Types

★ DATA TYPES IN JAVA

There are two types of data types in Java.



1. PRIMITIVE DATA TYPES

Primitive data types are the basic building blocks in Java.

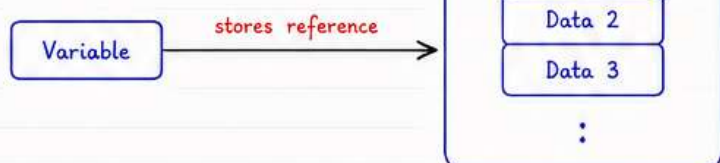
These types store actual values.

Type	Size	Default Value	Range / Description
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	-2^{31} to $2^{31} - 1$ (-2,147,483,648 to 2,147,483,647)
long	8 bytes	0L	-2^{63} to $2^{63} - 1$ (Very large range)
float	4 bytes	0.0f	Decimal numbers (6-7 digits precision)
double	8 bytes	0.0d	Decimal numbers (15-16 digits precision)
char	2 bytes	'\u0000'	0 to 65,535 (Unicode characters)
boolean	1 bit	false	true or false

2. NON-PRIMITIVE (REFERENCE) DATA TYPES

Non-primitive data types are used to store collections of data and more complex entities.

- String
- Arrays
- Classes
- Interfaces
- Collection Framework



Note : Java is a strongly typed language, meaning every variable must be declared with a specific type, and that type cannot be changed.

≡ JAVA FUNDAMENTALS NOTES ≡

2. OPERATORS IN JAVA

Key Point

Operators work on operands (variables or values) and produce a result.

Operators are special symbols that are used to perform **operations** on variables and values.

1. ARITHMETIC OPERATORS

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (Remainder)	10 % 3	1

Note : Division (/) between two integers gives integer result.
For decimal result, at least one operand should be **float/double**.

2. RELATIONAL (COMPARISON) OPERATORS

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or Equal to	5 >= 5	true
<=	Less than or Equal to	5 <= 4	false

Note : Relational operators return **boolean** values (true or false).

3. LOGICAL OPERATORS

Operator	Name	Meaning	Example	Result
&&	Logical AND	true if both conditions are true	(5 > 3) && (2 < 4)	true
	Logical OR	true if at least one condition is true	(5 > 3) (2 > 4)	true
!	Logical NOT	Reverses the result (true → false)	!(5 > 3)	false

Note : In Java, && and || use **short-circuit evaluation**.
• For **A && B** → If A is **false**, B is not evaluated.
• For **A || B** → If A is **true**, B is not evaluated.

4. ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5

Note : '=' is used to assign value.
Others are shorthand assignment operators.

5. UNARY OPERATORS

a) Increment (++)

Used to increase the value by 1.

- Pre-increment : ++x
- Post-increment : x++

Example	Operation	Result
int x = 5; ++x;	Pre-increment	x = 6
int x = 5; x++;	Post-increment	x = 6 (but uses 5 first)

b) Decrement (--)

Used to decrease the value by 1.

- Pre-decrement : --x
- Post-decrement : x--

Example	Operation	Result
int x = 5; --x;	Pre-decrement	x = 4
int x = 5; x--;	Post-decrement	x = 4 (but uses 5 first)

6. TERNARY (CONDITIONAL) OPERATOR

A shorthand way of writing **if-else** statement.

Syntax : condition ? expression1 : expression2

If condition is **true**,
expression1 is executed.

If condition is **false**,
expression2 is executed.

Example :

```
int age = 20;  
String result = (age >= 18) ? "Adult" : "Minor";
```

Output : Adult

7. OPERATOR PRECEDENCE (Higher precedence operators are evaluated first)

Precedence	Operators	Category	Example	Evaluates As
1	()	Parentheses	(3 + 2) * 5	(3 + 2) * 5
2	++, --, !, ~	Unary	++a + b	(++a) + b
3	*, /, %	Arithmetic	a + b * c	a + (b * c)
4	+, -	Arithmetic	a - b + c	(a - b) + c
5	<, <=, >, >=, instanceof	Relational	a > b && c < d	(a > b) && (c < d)
6	==, !=	Equality	a == b c != d	(a == b) (c != d)
7	&&	Logical AND	a && b c	(a && b) c
8		Logical OR		
9	?:	Ternary	a > b ? x : y	(a > b) ? x : y
10	=, +=, -=, *=, /=, %=	Assignment	a += 5	a += 5

QUICK RECAP

- ✓ Arithmetic → +, -, *, /, %
- ✓ Relational → ==, !=, >, <, >=, <=
- ✓ Logical → &&, ||, !
- ✓ Assignment → =, +=, -=, *=, /=, %=
- ✓ Unary → ++, --, !
- ✓ Ternary → ? :

JAVA FUNDAMENTALS NOTES

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

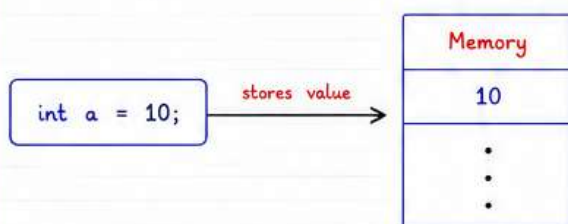
★ 1.1 PRIMITIVE DATA TYPES – MORE DETAILS

Primitive data types are predefined by Java and are not objects. They store single values and are more efficient.

Type	Size	Default Value	Details
byte	1 byte	0	Stores whole numbers from -128 to 127.
short	2 bytes	0	Stores whole numbers from -32,768 to 32,767.
int	4 bytes	0	Most commonly used for whole numbers. Range: -2^{31} to $2^{31} - 1$
long	8 bytes	0L	Used for very large whole numbers. Range: -2^{63} to $2^{63} - 1$
float	4 bytes	0.0f	Stores decimal numbers with single precision. Sufficient for 6-7 decimal digits.
double	8 bytes	0.0d	Stores decimal numbers with double precision. Sufficient for 15-16 decimal digits.
char	2 bytes	'\u0000'	Stores a single Unicode character. Example: 'A', '1', '@'
boolean	1 bit	false	Stores logical values: true or false.

★ MEMORY ALLOCATION (CONCEPT)

Primitive types store the actual value in memory.



★ SIZE COMPARISON (BYTES)

byte	short	int	long	float	double
1	2	4	8	4	8

Size increasing

★ TYPE CATEGORIES SUMMARY

Primitive Data Types

- Store single values.
- Actual value is stored in memory.
- Have fixed size.
- Examples: int, float, char, boolean, etc.

Non-Primitive (Reference) Data Types

- Store collections of values.
- Store reference (address) in memory.
- No fixed size.
- Examples: String, Arrays, Classes, etc.

Note : Choose the appropriate data type to save memory and improve performance.
For example, use **int** instead of **long** if the number is within **int** range.

JAVA FUNDAMENTALS NOTES

1. DATA TYPES IN JAVA (CONTINUED)

NON-PRIMITIVE (REFERENCE) DATA TYPES

Non-primitive data types are more complex and are used to store collections of values or user-defined data. They store **references** (addresses) in **stack** memory, while actual data is stored in **heap** memory.

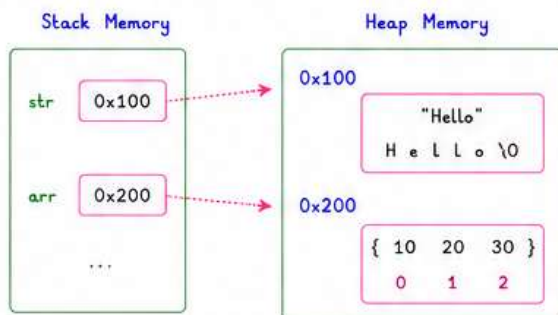
TYPES OF NON-PRIMITIVE DATA TYPES

Type	Description	Example	Default Value	Stored In	Example Code
String	Sequence of characters. Used to store text.	"Hello", "Java", "123"	null	Heap	String name = "Java"; String msg = new String("Hello"); System.out.println(name);
Array	Stores fixed-size sequence of elements of same type.	int[] arr = {1, 2, 3}; char[] ch = {'a', 'b'};	null	Heap	int[] arr = {10, 20, 30}; System.out.println(arr[1]); // 20
Class	User-defined blueprint to create objects.	Student s = new Student();	null	Heap	Student s = new Student("Ram", 21); s.display();
Interface	Reference type that contains abstract methods.	List<String> list;	null	Heap	List<String> list = new ArrayList<>(); list.add("A");
Enum	Special data type that consists of a fixed set of constants.	Day d = Day.MONDAY;	null	Heap	enum Day { MONDAY, TUESDAY, WEDNESDAY } Day d = Day.TUESDAY;

Key Points

- Default value of all reference types is null.
- Created using classes, constructors or literals.
- Support more functionality and are memory efficient for large data.

STACK VS HEAP MEMORY (REFERENCE TYPES)



Variables in stack hold reference (address) to actual data present in heap.

NULL IN REFERENCE TYPES

A reference variable can point to an object or be null.

Example :

```
String s = null; // s does not point to any object
int[] arr = null; // arr does not point to any array
Student s1 = null; // s1 does not point to any object
```



If you try to access a null reference, it causes **NullPointerException** at runtime.

EXAMPLE PROGRAM (USING NON-PRIMITIVE TYPES)

```
class Student {
    String name; // non-primitive (String)
    int age; // primitive

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println(name + " is " + age + " years old");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("John", 20); // class (object)
        s1.display();

        String city = "Delhi"; // String
        int[] marks = {85, 90, 78}; // Array
        System.out.println(city);
        System.out.println(marks[1]);
    }
}
```

Output :

```
John is 20 years old
Delhi
90
```

QUICK RECAP

- ✓ Non-primitive data types store references, not actual values.
- ✓ They are stored in **heap** memory.
- ✓ Default value of all non-primitive types is null.
- ✓ Examples : String, Array, Class, Interface, Enum.
- ✓ They provide more features and help in writing structured and reusable code.

WHEN TO USE?

- Use String to store text.
- Use Array to store multiple values of same type.
- Use Class to create objects for real-world entities.
- Use Interface for abstraction.
- Use Enum for fixed set of constants.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ LOOPS IN JAVA ≡

Loops are used to execute a block of code multiple times based on a condition.

Java provides 3 types of loops.

- for loop
- while loop
- do-while loop

Why Loops?

- To avoid repetition
- To iterate over a range
- To traverse arrays, strings, collections, etc.

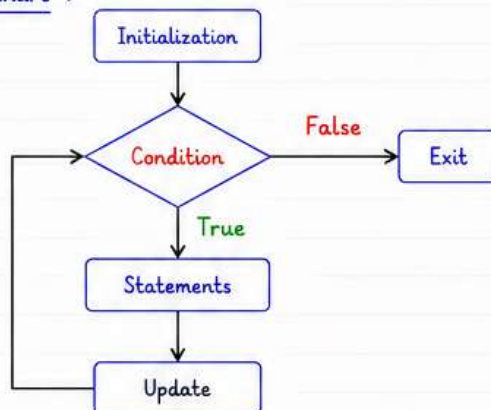
★ 1. FOR LOOP

The for loop is used when the number of iterations is known in advance.

Syntax :

```
for (initialization; condition; update) {  
    // statements  
}
```

Flowchart :



Example : Print numbers from 1 to 5

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

How it works?

- ① Initialize $i = 1$
- ② Check condition $i \leq 5 \rightarrow \text{True}$
- ③ Execute statements (print i)
- ④ Update $i \rightarrow i = i + 1$
- ⑤ Repeat steps ② to ④ until condition becomes **False**

Output :

```
1  
2  
3  
4  
5
```

★ COMPARISON OF LOOPS

Feature	for loop	while loop	do-while loop
When to use	When number of iterations is known	When number of iterations is unknown	When block must execute at least once
Condition Check	Before executing the loop	Before executing the loop	After executing the loop
Execution	0 or more times	0 or more times	1 or more times
Syntax	More compact	Less compact	Medium
Use Case	Counting, traversing, arrays, fixed iterations	User input validation, unknown iterations	Menu driven programs, atleast one execution

- Note :**
- Avoid infinite loops (condition should eventually become false).
 - Use loops carefully to improve performance and readability of code.

JAVA FUNDAMENTALS NOTES

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

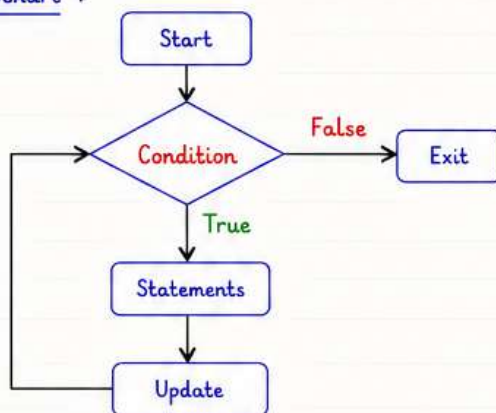
★ 2. WHILE LOOP

The **while** loop is used when the number of iterations is not known in advance.

Syntax :

```
while (condition) {  
    // statements  
    // update  
}
```

Flowchart :



Example : Print numbers from 1 to 5

```
int i = 1;  
while (i <= 5) {  
    System.out.println(i);  
    i = i + 1;  
}
```

How it works?

- ① Initialize $i = 1$
- ② Check condition $i \leq 5 \rightarrow \text{True}$
- ③ Execute statements (print i)
- ④ Update $i \rightarrow i = i + 1$
- ⑤ Repeat steps ② to ④ until condition becomes **False**

Output :

```
1  
2  
3  
4  
5
```

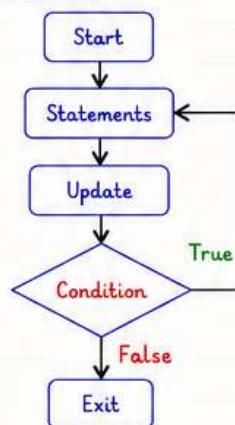
★ DO-WHILE LOOP

The **do-while** loop is similar to while loop, but the block of code is executed **at least once**.

Syntax :

```
do {  
    // statements  
    // update  
} while (condition);
```

Flowchart :



Example : Print numbers from 1 to 5

```
int i = 1;  
do {  
    System.out.println(i);  
    i = i + 1;  
} while (i <= 5);
```

Output :

```
1  
2  
3  
4  
5
```

★ COMPARISON OF WHILE vs DO-WHILE

Feature	while loop	do-while loop
Execution	May execute zero times	Executes at least once
Condition Check	Before executing the block	After executing the block
Syntax	while (condition) { ... }	do { ... } while (condition);
Use Case	When iterations may be zero	When at least one execution is required

Note : Use **while** when the condition is checked first.

Use **do-while** when the block must run at least once.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ LOOPS IN JAVA (CONTINUED) ≡

★ 3. FOR-EACH LOOP (ENHANCED FOR LOOP)

The for-each loop is used to iterate over arrays or collections easily.

Syntax :

```
for (datatype variable : array/collection) {  
    // statements  
    // variable takes each element  
}
```

Example 1 : Using array

```
int[] arr = {10, 20, 30, 40, 50};  
for (int num : arr) {  
    System.out.println(num);  
}
```

Output :

10 20 30 40 50

Example 2 : Using ArrayList

```
ArrayList<String> names = new ArrayList<>();  
names.add("Aman");  
names.add("Riya");  
names.add("Vikram");  
  
for (String name : names) {  
    System.out.println(name);  
}
```

Output :

Aman
Riya
Vikram

Important Points

- Works with arrays and collections.
- No need to use index.
- Read-only access (cannot modify elements directly).
- Cleaner and easier syntax.

★ COMPARISON: FOR LOOP vs FOR-EACH LOOP

Feature	For Loop	For-Each Loop (Enhanced For Loop)
Syntax	More verbose	Simpler and cleaner
Access	Provides index (can access position)	Does not provide index
Use Case	When index is needed or we want to modify elements	When we only need to read elements
Works With	Arrays, Collections, Strings	Arrays, Collections only
Modification	Can modify elements using index	Cannot modify elements directly
Flexibility	More flexible	Less flexible (read-only)

★ OTHER IMPORTANT LOOP NOTES

Break Statement

Used to exit the loop immediately.

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
    System.out.println(i);  
}
```

Output : 1 2

Continue Statement

Skips the current iteration and moves to next.

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    System.out.println(i);  
}
```

Output : 1 2 4 5

Nested Loops

A loop inside another loop.

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 2; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

Output :

* *
* *
* *

Note : Choose the right loop based on the requirement for better performance and readability of your code.



≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ BREAK and CONTINUE in LOOPS ≡

Both **break** and **continue** are used to control the flow of loops.

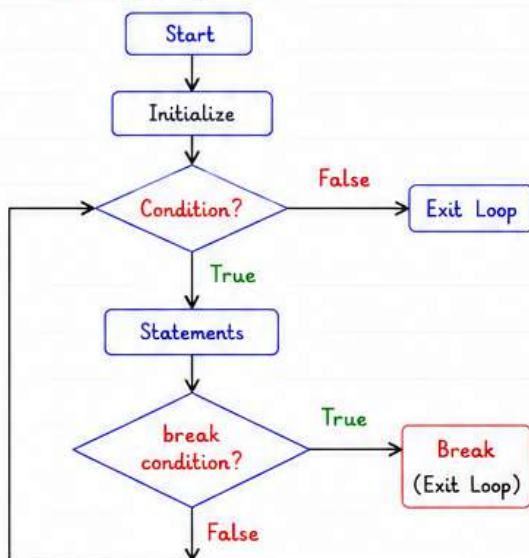
BREAK KEYWORD

- Terminates the loop immediately.
- Control comes out of the loop.
- Remaining iterations are skipped.

When to use?

Use **break** when a certain condition is met and we want to **exit the loop**.

Flowchart of break



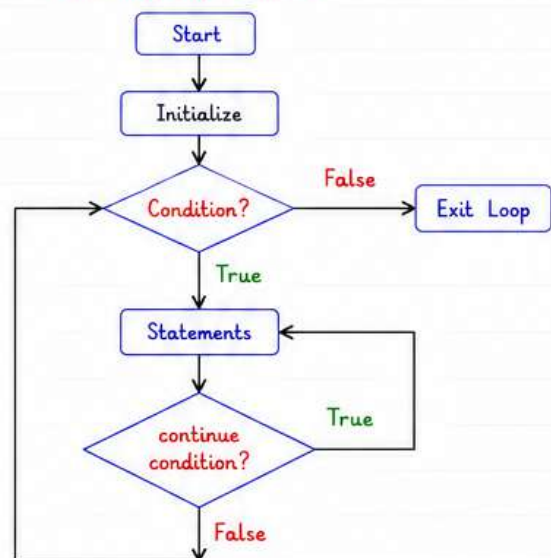
CONTINUE KEYWORD

- Skips the current iteration.
- Control moves to the next iteration.
- Remaining iterations are executed.

When to use?

Use **continue** when a certain condition is met and we want to **skip the current iteration**.

Flowchart of continue



EXAMPLE 1: Using break

```
// Print numbers from 1 to 10, but stop when number is 6
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break; // exit the loop
    }
    System.out.print(i + " ");
}
```

Output: 1 2 3 4 5

EXAMPLE 2: Using continue

```
// Print odd numbers from 1 to 10 (skip even numbers)
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // skip even numbers
    }
    System.out.print(i + " ");
}
```

Output: 1 3 5 7 9

WHEN TO USE?

Use **break** When

- We want to terminate the loop completely.
- A condition is met and further iterations are not required.
- Example: searching an element and found it.



Use **continue** When

- We want to skip some iterations.
- A condition is met, but remaining iterations are still needed.
- Example: skipping even numbers, invalid inputs, etc.

≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ CONDITIONAL STATEMENTS IN JAVA ≡

Conditional statements are used to make decisions in a program based on certain conditions.

Java provides the following conditional statements:

- if statement
- if-else statement
- else-if ladder
- nested if statement
- switch statement

Key Points

- Conditions must evaluate to **boolean** (true or false).
- If a condition is **true**, the corresponding block is executed.
- If a condition is **false**, the next possible condition is checked.
- If no condition is **true**, **else** block (if present) is executed.

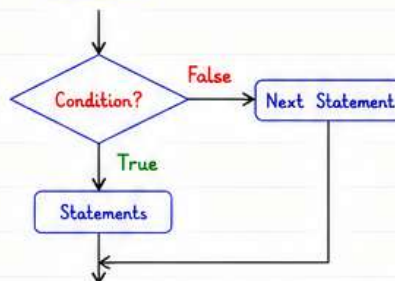
★ 1. if STATEMENT

Executes a block of code if the condition is true.

Syntax :

```
if (condition) {  
    // statements  
}
```

Flowchart :



Example :

```
int age = 18;  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
}
```

Output :

Eligible to vote

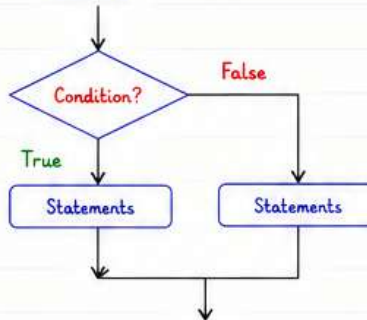
★ 2. if-else STATEMENT

Executes one block if condition is true otherwise executes another block.

Syntax :

```
if (condition) {  
    // statements  
} else {  
    // statements  
}
```

Flowchart :



Example :

```
int age = 16;  
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Output :

Minor

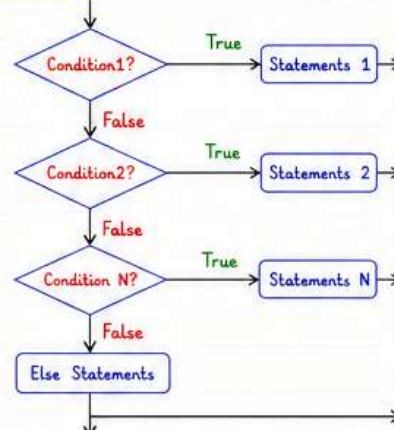
★ 3. else-if LADDER

Used to check multiple conditions.

Syntax :

```
if (condition1) {  
    // statements  
} else if (condition2) {  
    // statements  
} else if (condition3) {  
    // statements  
} else {  
    // statements  
}
```

Flowchart :



Example :

```
int marks = 75;  
if (marks >= 90) {  
    System.out.println("Grade A");  
} else if (marks >= 75) {  
    System.out.println("Grade B");  
} else if (marks >= 50) {  
    System.out.println("Grade C");  
} else {  
    System.out.println("Fail");  
}
```

Output :

Grade B

Note : Only the first **true** condition block is executed.
Once a block is executed, the remaining conditions are skipped.



≡ JAVA FUNDAMENTALS NOTES ≡

1. DATA TYPES, LOOPS, CONDITIONAL STATEMENTS

≡ CONDITIONAL STATEMENTS IN JAVA (CONTINUED) ≡

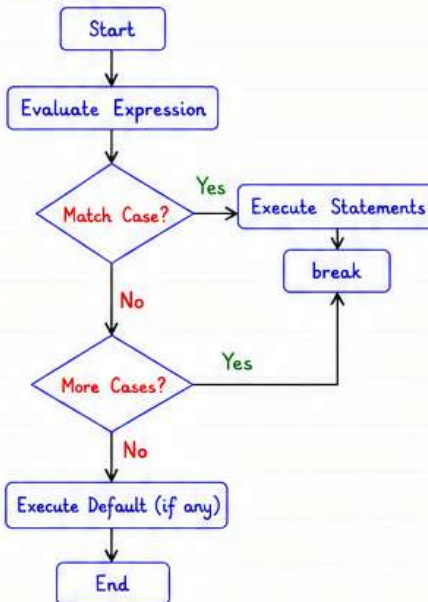
★ 4. SWITCH STATEMENT

The `switch` statement is used to execute one block of code from multiple alternatives.

Syntax :

```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    ...  
    default:  
        // statements  
}
```

Flowchart :



Example :

```
int day = 3;  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Invalid day");  
}
```

Output :

Wednesday

IMPORTANT POINTS ABOUT SWITCH

- The expression must be of type: byte, short, int, char, String, enum.
- Each case value must be unique.
- `break` is important to exit the switch. Without `break`, execution falls through to the next case.
- `default` is optional and executed when no case matches.
- Since Java 14, switch can be used as an expression (new feature).

FALL-THROUGH EXAMPLE (WITHOUT BREAK)

```
int num = 2;  
switch (num) {  
    case 1: System.out.println("One");  
    case 2: System.out.println("Two");  
    case 3: System.out.println("Three");  
    default: System.out.println("Other");  
}
```

Output :

Two
Three
Other

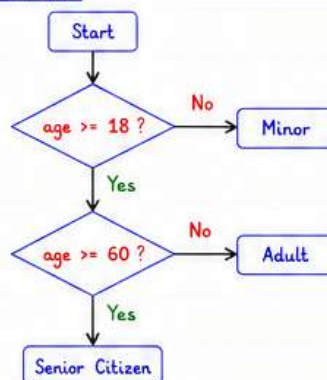
★ 5. NESTED IF STATEMENT

An if statement inside another if or else block.

Example :

```
int age = 20;  
if (age >= 18) {  
    if (age >= 60) {  
        System.out.println("Senior Citizen");  
    } else {  
        System.out.println("Adult");  
    }  
} else {  
    System.out.println("Minor");  
}
```

Flowchart :



QUICK RECAP

- ☑ if → for one condition
- ☑ if-else → for two conditions
- ☑ else-if ladder → for multiple conditions
- ☑ switch → for multiple fixed values
- ☑ Nested if → for complex conditions



Note : Use if-else when conditions are range based or complex.

Use switch when you have many fixed choices of the same variable.

COMMENT PDF

— TO GET —

COMPLETE PDF

OF

COLLECTION

JAVA COLLECTIONS FRAMEWORK

TYPES OF COLLECTIONS

-  LIST
-  SET
-  MAP
-  QUEUE
-  STACK
-  COMPARATOR & COMPARABLE

WHY COLLECTIONS ?

-  EFFICIENT DATA HANDLING
-  REUSABILITY
-  BETTER PERFORMANCE
-  DATA STRUCTURE SUPPORT
-  INTEROPERABILITY
-  SCALABILITY



SAVE



SHARE



FOLLOW



aman_views

